Sztuczna inteligencja i inżynieria wiedzy
Lista 2

Jan Jakubik, Piotr Syga, Daria Dziubałtowska, Michał Czuba

Marzec 2026

1 Cel listy

Celem ćwiczenia jest praktyczne zapoznanie się z algorytmami grania w gry dwuosobowe - algorytmem Minimax oraz jego ulepszeniem z wykorzystaniem alfa-beta cięć.

1.1 Definicje

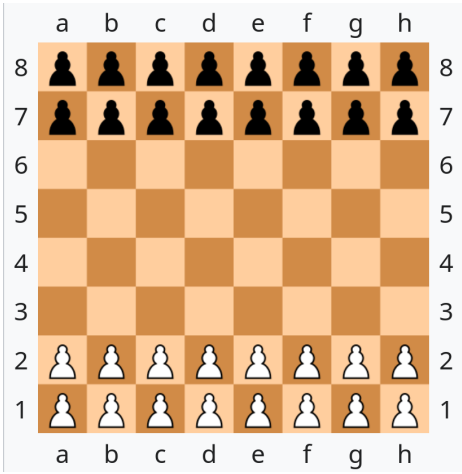
Algorytm Minimax jest strategią przeszukiwania drzewa gry, która służy do podejmowania decyzji w grach dwuosobowych o sumie zerowej, takich jak szachy, warcaby czy go. Algorytm ten dąży do minimalizacji maksymalnej możliwej straty, której gracz może doznać, biorąc pod uwagę ruchy przeciwnika.

Algorytm Minimax opiera się na przeglądaniu drzewa gry, gdzie każdy wierzchołek drzewa reprezentuje jej stan, a krawędzie pomiędzy nimi reprezentują ruchy graczy. Liściami będą wierzchołki reprezentujące zakończenie gry — wygraną jednego z graczy. W praktyce, ze względu na liczbę wierzchołków wzrastającą wykładniczo z głębokością stosujemy **limit głębokości** oraz **heurystyki** oceny stanu gry, pozwalające nam ocenić niezakończoną grę w przypadku dojścia do tego limitu.

Alfa-beta cięcie. Algorytm Minimax można ulepszyć za pomocą alfa-beta cięcia, które polega na zatrzymaniu rekurencyjnego przeglądania pewnej części drzewa, gdy określone warunki pozwalają na stwierdzenie, że dalsze przeglądanie nie ma już sensu.

2 Zadania

Breakthrough to gra planszowa zaprojektowana przez Dan Troyka w 2000 roku. Rozgrywka dla dwóch graczy toczy się na kwadratowej planszy typu szachownica o domyślnym rozmiarze 8x8 pól. Każdy z graczy ma swój własny kolor, standardowo (biały i czarny). Na starcie każdy z graczy zapełnia dwa skrajne wiersze swoimi pionami, tak jak na Ryc. 1. Grę rozpoczyna gracz biały.



Rysunek 1: Breakthrough – wstępne ustawienie planszy.
Source: Źródło: `https://en.wikipedia.org/wiki/Breakthrough_(board_game)`

Gracze poruszają się naprzemiennie. Gracz może przesunąć jeden ze swoich pionków o jedno pole prosto, bądź po skosie, jeśli pole jest puste. Gracz może zbić sąsiadujący pionek przeciwnika i zająć jego miejsce tylko jeśli pion przeciwnika znajduje się po skosie od pionka gracza. Nie ma możliwości bicia, gdy pionki graczy znajdują się na wprost siebie. Wygrywa gracz, którego pionek jako pierwszy stanie na przeciwnej krawędzi planszy. Przykładowo, wg. Ryc. 1, gracz biały musi dotrzeć na dowolne pole z wiersza ósmego, aby wygrać grę.

Korzystając z wiadomości podanych na tej liście oraz na wykładzie, napisz program, który gra w grę Breakthrough (rozmiar planszy $n \times m = 8 \times 8$, można modyfikować). Program powinien na standardowym wejściu przyjmować planszę startową (np. domyślną startową dla gry Breakthrough), pozwalać na wybór heurystyki oraz dostosowanie parametru d, będącego maksymalną głębokością przeszukiwania w drzewie decyzyjnym. Reprezentacja planszy ma mieć postać m linii składających się z n oddzielonych spacjami liczbami – `B` oznacza krążek gracza pierwszego, `W` oznacza krążek gracza drugiego, `_` oznacza pole puste, `o` oznacza pole z którego wykonany został ostatni ruch.

Następuje tura gracza pierwszego. Po uruchomieniu, program powinien:

- Wersja podstawowa – rozegrać całą grę przy użyciu algorytmu Minimax z perspektywy gracza pierwszego. Gracz drugi zawsze wykonuje optymalny ruch (obaj gracze używają tych samych heurystyk, więc przewidywania gracza pierwszego, co do ruchu przeciwnika są zawsze trafne).
- Wersja rozszerzona (na dodatkowe 20 pkt.) – program zezwoli na grę między dwoma odrębnymi agentami (oponent nie zawsze musi działać optymalnie, gracze mogą zmieniać strategię).

Na standardowym wyjściu program powinien zwrócić reprezentację planszy po zakończeniu rozgrywki. Następnie w dodatkowej linii powinna być wypisana informacja o liczbie rund oraz o tym, który z graczy wygrał. Na standardowym wyjściu błędów powinna zostać zwrócona liczba odwiedzonych węzłów drzewa decyzyjnego oraz czas działania algorytmu.

Punktacja:

1. Poprawne zdefiniowanie stanu gry i funkcji generującej możliwe ruchy dla danego stanu i gracza. (10 punktów)
2. Zbudowanie **zbioru heurystyk** oceny stanu gry dla każdego z graczy, każdy z graczy powinien mieć przynajmniej 3 różne strategie. (20 punktów)
3. Implementacja metody **Minimax** i programu w wersji podstawowej (30 punktów)
4. Implementacja **alfa-beta** w wersji podstawowej. (40 punktów)
5. Modyfikacja programu do wersji rozszerzonej, co umożliwi rozegranie partii pomiędzy dwoma programami (np. dwoma wywołaniami tej samej aplikacji) oraz zastosowanie heurystyk adaptacyjnie zmieniających strategię gracza w celu osiągnięcia zwycięstwa (**dodatkowe** 20 punktów).

Do zadania przygotuj raport zawierający opis teoretyczny metody, formalne sformułowanie problemu z ograniczeniami, stanu gry i drzewa decyzyjnego oraz opis idei rozwiązania wraz z krótkim przykładem. W raporcie wskaż wykorzystane materiały źródłowe oraz krótko opisz biblioteki wykorzystane przy implementacji. W podsumowaniu raportu dodatkowo opisz napotkane problemy implementacyjne przy wykonywaniu zadania. Raport wyślij prowadzącemu przynajmniej na 24 godziny przed oddaniem listy.

3 Literatura

- Breakthrough - Wikipedia
- Breakthrough - opis zasad
- J.F. Nordstrom – Introduction to Game Theory
- M. Maschler et al., Game Theory
- N. Nisan et al., Algorithmic Game Theory
- M. J. Osborne, An Introduction to Game Theory